



|                      |                             |
|----------------------|-----------------------------|
| <b>Student Name</b>  | <b>Sarvesh Naik</b>         |
| <b>SRN No</b>        | 31241943                    |
| <b>Roll No</b>       | 33                          |
| <b>Program</b>       | Computer Science Engg.      |
| <b>Year</b>          | Third Year                  |
| <b>Division</b>      | F                           |
| <b>Subject</b>       | Artificial Intelligence Lab |
| <b>Assignment No</b> | 6                           |

## Assignment 6

**Problem statement: Implement Constraint Satisfaction Algorithm for the following problems: a. Cryptarithmic b. Crossword puzzle c. Map coloring problem**

### Objective:

The objective of this assignment is to understand and implement Constraint Satisfaction Problems (CSP) using different real-world problem scenarios. This includes developing algorithms to solve problems such as cryptarithmic puzzles, crossword puzzles, and map coloring, where solutions must satisfy a set of constraints.

### a. Cryptarithmic

#### Theory:

Cryptarithmic is a type of **Constraint Satisfaction Problem (CSP)** where letters are replaced by digits to make a valid arithmetic equation. Each letter represents a unique digit (0–9), and no two letters can have the same value. The solution must satisfy the given arithmetic operation (like addition or multiplication).

For example:

$$\text{SEND} + \text{MORE} = \text{MONEY}$$

Constraints involved:

- Each letter has a unique digit.
- Leading digits cannot be zero.
- The arithmetic equation must hold true.

This problem is typically solved using **backtracking**, where possible digit assignments are tried and invalid ones are discarded based on constraints.

#### Algorithm:

1. Identify all unique letters in the equation.
2. Assign digits (0–9) to each letter.
3. Ensure no two letters have the same digit.
4. Apply constraints:
  - Leading letters  $\neq 0$
  - Arithmetic equation must be satisfied
5. Use backtracking:
  - Assign a digit to a letter.
  - Check if constraints are satisfied.
  - If not, backtrack and try another digit.
6. Repeat until a valid solution is found.
7. Display the solution.

## Code:

```
"""
=====

Constraint Satisfaction Problem (CSP)

Problem : CRYPTARITHMETIC

Example : SEND + MORE = MONEY

=====
"""

import os

import itertools

import matplotlib.pyplot as plt

import matplotlib.gridspec as gridspec

from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

import matplotlib.path as pe

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

OUTPUT_FILE = os.path.join(SCRIPT_DIR, "csp_cryptarithmic.png")

# — Colour palette —

CLR_BG = "#0D1117"

CLR_PANEL = "#161B22"

CLR_ACCENT = "#58A6FF"

CLR_GREEN = "#3FB950"

CLR_RED = "#F85149"

CLR_YELLOW = "#D29922"

CLR_TEXT = "#C9D1D9"

CLR_DIM = "#8B949E"

CLR_BORDER = "#30363D"
```

```

CLR_HEADER      = "#58A6FF"

# =====
#  CRYPTARITHMETIC SOLVER (CSP with Backtracking)
#  =====

def solve_cryptarithmic(word1, word2, word3):
    """
    Solve: word1 + word2 = word3
    Returns all solutions and the search tree nodes (for visualization).
    """
    letters = list(set(word1 + word2 + word3))
    leading = {word1[0], word2[0], word3[0]}
    digits = list(range(10))

    solutions = []
    tree_nodes = [] # (assignment_so_far, depth, result)
    node_count = [0]

    def word_val(word, assign):
        val = 0
        for ch in word:
            val = val * 10 + assign[ch]
        return val

    def backtrack(assign, remaining, depth):
        node_count[0] += 1

        snapshot = dict(assign)

        tree_nodes.append((snapshot, depth, None)) # None = pending

```

```

    if not remaining:

        v1 = word_val(word1, assign)

        v2 = word_val(word2, assign)

        v3 = word_val(word3, assign)

        ok = (v1 + v2 == v3)

        tree_nodes[-1] = (snapshot, depth, ok)

        if ok:

            solutions.append((dict(assign), v1, v2, v3))

    return

letter = remaining[0]

used = set(assign.values())

for d in digits:

    if d in used:

        continue

    if d == 0 and letter in leading:

        continue

    assign[letter] = d

    backtrack(assign, remaining[1:], depth + 1)

    del assign[letter]

backtrack({}, letters, 0)

return solutions, tree_nodes, node_count[0]

```

```
# =====
```

```
# TERMINAL OUTPUT
```

```
# =====

def print_results(word1, word2, word3, solutions, node_count):

    # Fixed-width box - wide enough for any input

    W = 62 # inner content width (between the | pipes)

    def pad(text):

        """Left-align text and pad to exactly W chars."""

        return text[:W].ljust(W)

    def bar(c="="):

        return "+" + c * W + "+"

    problem_line = f"Problem : {word1} + {word2} = {word3}"
    vars_line = f"Variables : {' , '.join(sorted(set(word1+word2+word3)))}"
    constr_line = f"Constraint : {word1} + {word2} = {word3} (no leading zeros)"
    nodes_line = f"Nodes explored : {node_count}"
    solns_line = f"Solutions found: {len(solutions)}"
    status_line = f"Status : {'SOLVED ✓' if solutions else 'NO SOLUTION FOUND X'}"

    print()
    print(bar())
    print(f"|{'CSP -- CRYPTARITHMETIC SOLVER':^{W}}|")
    print(bar())
    print(f"| {pad(problem_line)} |")
    print(bar("-"))
    print(f"| {pad(vars_line)} |")
    print(f"| {pad(constr_line)} |")
    print(bar("-"))
```

```

print(f"| {pad(nodes_line)}|")

print(f"| {pad(solns_line)}|")

print(f"| {pad(status_line)}|")

print(bar())

if not solutions:

    return

print(f"\n {'Step':<6} {'Assignment':<40} {'Equation':<30} {'Status'}")
print(" " + "=" * 90)

for i, (assign, v1, v2, v3) in enumerate(solutions, 1):

    asgn_str = " ".join(f"{k}={v}" for k, v in sorted(assign.items()))

    eq_str = f"{v1} + {v2} = {v3}"

    print(f" {i:<6} {asgn_str:<40} {eq_str:<30} ✓ VALID")

print()

# =====
# VISUALIZATION
# =====

def draw_letter_box(ax, x, y, letter, digit, size=0.7, highlight=False):

    color = CLR_GREEN if highlight else CLR_PANEL
    edge = CLR_GREEN if highlight else CLR_ACCENT

    ax.add_patch(FancyBboxPatch(

        (x, y), size, size,

        boxstyle="round,pad=0.05",

        facecolor=color, edgecolor=edge, linewidth=1.5, zorder=3

    ))

```

```

ax.text(x + size/2, y + size*0.72, letter,

        ha="center", va="center", fontsize=9,

        color=CLR_DIM, fontfamily="monospace", zorder=4)

ax.text(x + size/2, y + size*0.28, str(digit) if digit is not None else "?",

        ha="center", va="center", fontsize=13, fontweight="bold",

        color=CLR_TEXT if digit is not None else CLR_DIM,

        fontfamily="monospace", zorder=4)

def draw_word(ax, word, assignment, start_x, y, label=None):

    for i, ch in enumerate(word):

        d = assignment.get(ch)

        draw_letter_box(ax, start_x + i * 0.85, y, ch, d)

    if label:

        ax.text(start_x - 0.5, y + 0.35, label,

                ha="center", va="center", fontsize=11,

                color=CLR_ACCENT, fontfamily="monospace", fontweight="bold")

def visualize(word1, word2, word3, solutions, tree_nodes, node_count):

    fig = plt.figure(figsize=(18, 12), facecolor=CLR_BG)

    fig.suptitle(

        f"CSP - Cryptarithmic [{word1} + {word2} = {word3}]\n"

        "Technique: Backtracking with Constraint Propagation",

        color=CLR_HEADER, fontsize=15, fontweight="bold",

        fontfamily="monospace", y=0.98

    )

    gs = gridspec.GridSpec(2, 3, figure=fig,

```

```

        hspace=0.55, wspace=0.35,

        left=0.04, right=0.97, top=0.90, bottom=0.06)

# — Panel 1: Problem Statement —————

ax1 = fig.add_subplot(gs[0, 0])

ax1.set_facecolor(CLR_PANEL)

ax1.set_xlim(0, 5)

ax1.set_ylim(0, 4)

ax1.axis("off")

best = solutions[0] if solutions else {}

assign = best[0] if best else {}

max_len = max(len(word1), len(word2), len(word3))

ox = (5 - max_len * 0.85) / 2

draw_word(ax1, word1, assign, ox + (max_len - len(word1)) * 0.85, 3.0)

draw_word(ax1, word2, assign, ox + (max_len - len(word2)) * 0.85, 2.1)

ax1.plot([ox - 0.1, ox + max_len * 0.85], [1.9, 1.9], color=CLR_ACCENT, lw=1.5)

ax1.text(ox - 0.3, 2.1, "+", color=CLR_ACCENT, fontsize=14, fontweight="bold",

        fontfamily="monospace")

draw_word(ax1, word3, assign, ox + (max_len - len(word3)) * 0.85, 1.0)

ax1.set_title("Problem Statement", color=CLR_ACCENT, fontsize=10,

        fontfamily="monospace", pad=6)

if solutions:

    v1, v2, v3 = solutions[0][1], solutions[0][2], solutions[0][3]

    ax1.text(2.5, 0.3, f"{v1} + {v2} = {v3}", ha="center",

            color=CLR_GREEN, fontsize=11, fontfamily="monospace", fontweight="bold")

```

```

# — Panel 2: Variable - Digit Assignment Table —————

ax2 = fig.add_subplot(gs[0, 1])

ax2.set_facecolor(CLR_PANEL)

ax2.axis("off")

ax2.set_title("Variable → Digit Assignment", color=CLR_ACCENT,
              fontsize=10, fontfamily="monospace", pad=6)

if solutions:

    assign_map = solutions[0][0]

    letters_sorted = sorted(assign_map.keys())

    col_labels = ["Letter", "Digit", "Role"]

    leading = {word1[0], word2[0], word3[0]}

    table_data = []

    for ltr in letters_sorted:

        role = "Leading" if ltr in leading else "Middle/End"

        table_data.append([ltr, str(assign_map[ltr]), role])

    tbl = ax2.table(

        cellText=table_data,

        colLabels=col_labels,

        cellLoc="center", loc="center",

        bbox=[0.05, 0.05, 0.9, 0.85]

    )

    tbl.auto_set_font_size(False)

    tbl.set_fontsize(10)

    for (r, c), cell in tbl.get_celld().items():

        cell.set_facecolor(CLR_BG if r == 0 else CLR_PANEL)

```

```

        cell.set_edgecolor(CLR_BORDER)

        cell.set_text_props(
            color=CLR_ACCENT if r == 0 else CLR_TEXT,
            fontfamily="monospace",
            fontweight="bold" if r == 0 else "normal"
        )
    else:
        ax2.text(0.5, 0.5, "No Solution Found", ha="center", va="center",
            color=CLR_RED, fontsize=12, fontfamily="monospace")

# — Panel 3: Stats & Constraints —————
ax3 = fig.add_subplot(gs[0, 2])
ax3.set_facecolor(CLR_PANEL)
ax3.axis("off")
ax3.set_title("CSP Statistics", color=CLR_ACCENT,
    fontsize=10, fontfamily="monospace", pad=6)

all_letters = sorted(set(word1 + word2 + word3))

stats = [
    ("Variables",    str(len(all_letters))),
    ("Domain",      "{ 0 - 9 }"),
    ("Constraints",  "Alldiff + Sum"),
    ("Nodes Explored", str(node_count)),
    ("Solutions",    str(len(solutions))),
    ("Status",       "SOLVED ✓" if solutions else "NO SOL ✗"),
]

for i, (label, val) in enumerate(stats):
    y_pos = 0.88 - i * 0.14

```

```

        color = CLR_GREEN if "SOLVED" in val else (CLR_RED if "NO SOL" in val else
CLR_TEXT)

        ax3.text(0.05, y_pos, f"► {label}:", color=CLR_DIM,
                 fontsize=9, fontfamily="monospace", transform=ax3.transAxes)

        ax3.text(0.60, y_pos, val, color=color,
                 fontsize=9, fontfamily="monospace", fontweight="bold",
                 transform=ax3.transAxes)

# — Panel 4: Search Tree (partial, first 60 nodes) —————

ax4 = fig.add_subplot(gs[1, :])

ax4.set_facecolor(CLR_BG)

ax4.axis("off")

ax4.set_title("Backtracking Search Tree (first nodes per depth level)",
              color=CLR_ACCENT, fontsize=10, fontfamily="monospace", pad=6)

# Group nodes by depth — show up to 8 per depth, up to depth 5

depth_groups = {}

for snap, depth, result in tree_nodes:

    if depth not in depth_groups:

        depth_groups[depth] = []

    if len(depth_groups[depth]) < 8:

        depth_groups[depth].append((snap, result))

max_depth = min(max(depth_groups.keys()) if depth_groups else 0, 5)

ax4.set_xlim(0, 1)

ax4.set_ylim(-0.1, 1.1)

node_positions = {}

for depth in range(max_depth + 1):

    nodes = depth_groups.get(depth, [])

```

```

n = len(nodes)

for j, (snap, result) in enumerate(nodes):

    x = (j + 0.5) / max(n, 1)

    y = 1.0 - depth * (1.0 / max(max_depth, 1))

    if result is True:

        clr, ec = CLR_GREEN, CLR_GREEN

    elif result is False:

        clr, ec = "#2D1B1B", CLR_RED

    else:

        clr, ec = CLR_PANEL, CLR_ACCENT

    key = f"{depth}-{j}"

    node_positions[key] = (x, y)

    circle = plt.Circle((x, y), 0.025, color=clr, ec=ec, lw=1.2, zorder=3)

    ax4.add_patch(circle)

    label = ""

    if snap:

        last_key = list(snap.keys())[-1] if snap else ""

        last_val = snap.get(last_key, "")

        label = f"{last_key}={last_val}" if last_key else ""

    ax4.text(x, y - 0.055, label, ha="center", va="top",

             fontsize=6, color=CLR_DIM, fontfamily="monospace")

    # Connect to parent

    if depth > 0 and depth_groups.get(depth - 1):

        parent_n = len(depth_groups[depth - 1])

```

```

        parent_j = min(j, parent_n - 1)

        px = (parent_j + 0.5) / max(parent_n, 1)

        py = 1.0 - (depth - 1) * (1.0 / max(max_depth, 1))

        ax4.plot([px, x], [py, y], color=CLR_BORDER, lw=0.6, zorder=1)

# Depth labels
for depth in range(max_depth + 1):

    y = 1.0 - depth * (1.0 / max(max_depth, 1))

    ax4.text(0.01, y, f"D{depth}", ha="left", va="center",

             fontsize=7, color=CLR_DIM, fontfamily="monospace")

# Legend
for lbl, clr in [("Solution", CLR_GREEN), ("Failure", CLR_RED), ("Pending",
CLR_ACCENT)]:

    circle = plt.Circle((0, 0), 0.01, color=clr)

    ax4.text(0.78, -0.07, "● Solution ", color=CLR_GREEN, fontsize=8,
fontfamily="monospace")

    ax4.text(0.87, -0.07, "● Failure ", color=CLR_RED, fontsize=8,
fontfamily="monospace")

    ax4.text(0.95, -0.07, "● Node", color=CLR_ACCENT, fontsize=8, fontfamily="monospace")

# Status banner
stat_clr = CLR_GREEN if solutions else CLR_RED

stat_txt = (f"✓ SOLVED - {len(solutions)} solution(s) found | "

            f"{node_count} nodes explored via Backtracking")

if not solutions:

    stat_txt = f"✗ NO SOLUTION FOUND - {node_count} nodes explored"

fig.text(0.5, 0.013, stat_txt, ha="center", va="bottom",

         fontsize=10, fontweight="bold", fontfamily="monospace",

         color=stat_clr,

```

```

        bbox=dict(boxstyle="round,pad=0.4",
                    facecolor="#0D2818" if solutions else "#2D1B1B",
                    edgecolor=stat_clr, lw=1.5))

plt.savefig(OUTPUT_FILE, dpi=160, bbox_inches="tight", facecolor=CLR_BG)

print(f"\n [INFO] Visualization saved -> {OUTPUT_FILE}")

plt.show()

# =====
# USER INPUT
# =====

def get_words():

    print()

    print("+" + "=" * 62 + "+")

    print("| CSP -- Cryptarithmic Solver |")

    print("| Solves : WORD1 + WORD2 = WORD3 |")

    print("+" + "=" * 62 + "+")

    print()

    print(" Press Enter to use default [ SEND + MORE = MONEY ]")

    choice = input(" or type custom to enter your own words : ").strip().lower()

    if choice == "custom":

        print()

        word1 = input(" Enter WORD1 : ").strip().upper()

        word2 = input(" Enter WORD2 : ").strip().upper()

        word3 = input(" Enter WORD3 (=result) : ").strip().upper()

        # Validate only alpha

```

```

        for w in [word1, word2, word3]:

            if not w.isalpha():

                print("  [!] Only alphabetic characters allowed. Using default.")

                return "SEND", "MORE", "MONEY"

            return word1, word2, word3

    return "SEND", "MORE", "MONEY"

# =====
#  MAIN
#  =====

def main():

    word1, word2, word3 = get_words()

    all_letters = set(word1 + word2 + word3)

    print(f"\n  Problem    : {word1} + {word2} = {word3}")

    print(f"  Variables : {' ', ' '.join(sorted(all_letters))}")

    print(f"  Solving via Backtracking CSP ... \n")

    solutions, tree_nodes, node_count = solve_cryptarithmic(word1, word2, word3)

    print_results(word1, word2, word3, solutions, node_count)

    visualize(word1, word2, word3, solutions, tree_nodes, node_count)

if __name__ == "__main__":

    main()

```

## Output :

```
(ai_env) sarveshmac@Sarveshs-MacBook-Air Artificial Intelligence % python Assignment-6A.py

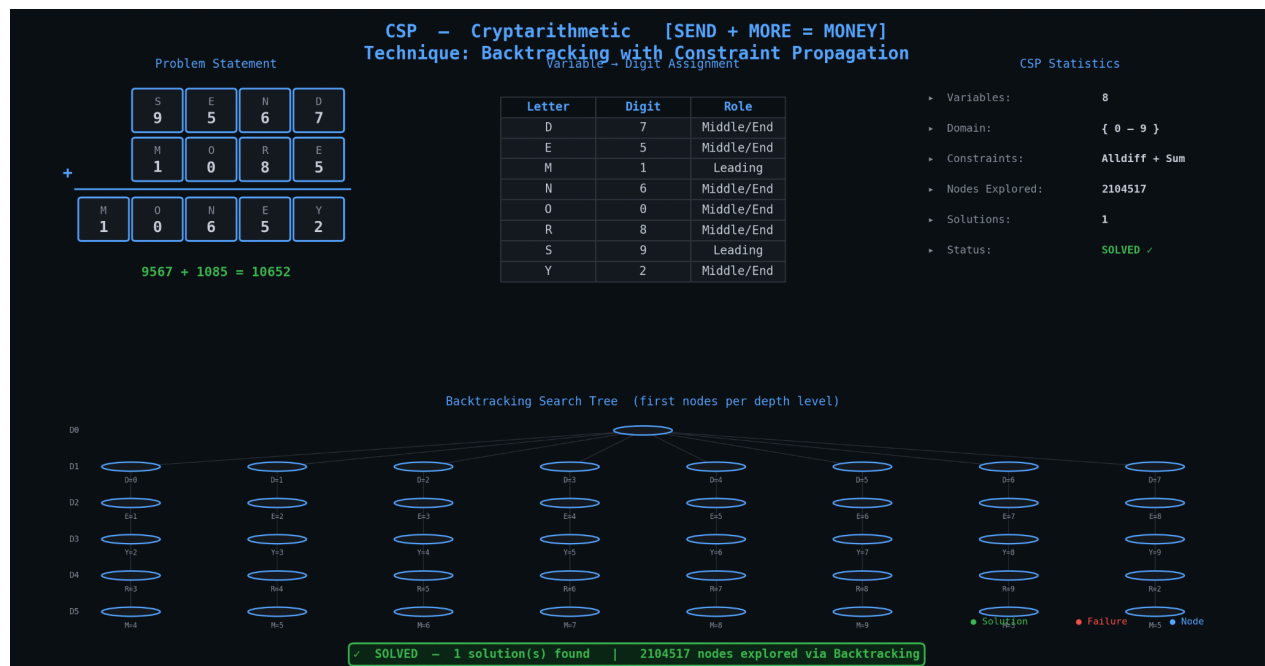
=====
CSP -- Cryptarithmic Solver
Solves : WORD1 + WORD2 = WORD3
=====

Press Enter to use default [ SEND + MORE = MONEY ]
or type custom to enter your own words :

Problem : SEND + MORE = MONEY
Variables : D, E, M, N, O, R, S, Y
Solving via Backtracking CSP ...

=====
CSP -- CRYPTARITHMETIC SOLVER
=====
Problem : SEND + MORE = MONEY
Variables : D, E, M, N, O, R, S, Y
Constraint : SEND + MORE = MONEY (no leading zeros)
=====
Nodes explored : 2104517
Solutions found: 1
Status : SOLVED ✓
=====

Step  Assignment                      Equation                      Status
=====
1      D=7 E=5 M=1 N=6 O=0 R=8 S=9 Y=2  9567 + 1085 = 10652          ✓ VALID
```



## b. Crossword puzzle

### Theory:

The crossword puzzle problem is a CSP where words must be placed in a grid such that all constraints are satisfied. Each slot (horizontal or vertical) must be filled with a valid word, and overlapping letters must match.

Constraints involved:

- Words must fit exactly in the given slots.
- Intersecting words must have matching characters.
- Each word is used only once.

This problem is solved using **backtracking with constraint checking**, ensuring that partial solutions remain valid.

### Algorithm:

Represent the crossword grid with empty and blocked cells.

Identify all word slots (horizontal & vertical).

Take a list of words to be filled.

Select a slot and try placing each word.

Check constraints:

- Word length matches slot length
- Overlapping letters match existing letters

If valid:

- Place the word
- Recur for next slot

If not valid:

- Try next word

If no word fits, backtrack.

Continue until all slots are filled.

Display the completed crossword.

### Code:



```

=====

Constraint Satisfaction Problem (CSP)

Problem : CROSSWORD PUZZLE

Technique: Backtracking + Arc Consistency (AC-3)

=====

"""

import os
import copy
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
OUTPUT_FILE = os.path.join(SCRIPT_DIR, "csp_crossword.png")

# — Colour palette —

CLR_BG          = "#0A0A0F"
CLR_PANEL       = "#12121A"
CLR_CELL_EMPTY = "#1A1A28"
CLR_CELL_BLOCK = "#050508"
CLR_ACCENT      = "#A78BFA"
CLR_GREEN       = "#34D399"
CLR_RED         = "#F87171"
CLR_YELLOW      = "#FBBF24"
CLR_TEXT        = "#E2E8F0"
CLR_DIM         = "#64748B"
CLR_BORDER      = "#2D2D3F"
CLR_HEADER      = "#A78BFA"

```

```

CLR_ACROSS      = "#60A5FA"

CLR_DOWN        = "#F472B6"


# =====

#  CROSSWORD DEFINITION

# =====

"""

Default 5x5 crossword grid (# = blocked):


    0    1    2    3    4
0 [ _  _  _  _  _ ]
1 [ _  #  _  #  _ ]
2 [ _  _  _  _  _ ]
3 [ _  #  _  #  _ ]
4 [ _  _  _  _  _ ]


Slots:

ACROSS:  1A (0,0)→len5    2A (1,0)len1+(1,2)len1+(1,4)len1 → simplified slots
          3A (2,0)→len5    4A (3, ...)    5A (4,0)→len5

DOWN:    1D (0,0)→len5    2D (0,2)→len5    3D (0,4)→len5

"""

DEFAULT_GRID = [

    [0, 0, 0, 0, 0],

    [0, -1, 0, -1, 0],

    [0, 0, 0, 0, 0],

    [0, -1, 0, -1, 0],


```

```

    [0, 0, 0, 0, 0],

] # 0 = open cell, -1 = blocked

DEFAULT_SLOTS = {

    "1-ACROSS": {"cells": [(0,0), (0,1), (0,2), (0,3), (0,4)], "length": 5},
    "3-ACROSS": {"cells": [(2,0), (2,1), (2,2), (2,3), (2,4)], "length": 5},
    "5-ACROSS": {"cells": [(4,0), (4,1), (4,2), (4,3), (4,4)], "length": 5},
    "1-DOWN":    {"cells": [(0,0), (1,0), (2,0), (3,0), (4,0)], "length": 5},
    "2-DOWN":    {"cells": [(0,2), (1,2), (2,2), (3,2), (4,2)], "length": 5},
    "3-DOWN":    {"cells": [(0,4), (1,4), (2,4), (3,4), (4,4)], "length": 5},

}

DEFAULT_WORDLIST = [

    "CRANE", "SHARE", "ALTAR", "CASTE", "RESTS",

    "CHORE", "ANGER", "NASAL", "TERSE", "EARTH",

    "STARS", "LATER", "SOLAR", "TREES", "RACES",

    "LANES", "RALES", "EARNS", "ARLES", "SNARE",

]

# =====

# CSP SOLVER (Backtracking + AC-3)

# =====

def get_constraints(slots):

    """Find all shared (slot_a, pos_a, slot_b, pos_b) intersections."""

    constraints = []

    slot_names = list(slots.keys())

    for i in range(len(slot_names)):

```

```

        for j in range(i + 1, len(slot_names)):

            sa, sb = slot_names[i], slot_names[j]

            cells_a = slots[sa]["cells"]

            cells_b = slots[sb]["cells"]

            for pa, cell in enumerate(cells_a):

                if cell in cells_b:

                    pb = cells_b.index(cell)

                    constraints.append((sa, pa, sb, pb))

    return constraints

def is_consistent(word_a, pos_a, word_b, pos_b):

    return word_a[pos_a] == word_b[pos_b]

def solve_crossword(slots, wordlist):

    constraints = get_constraints(slots)

    slot_names = list(slots.keys())

    domains = {s: [w for w in wordlist if len(w) == slots[s]["length"]]
                for s in slot_names}

    assignment = {}

    steps = [] # (assignment_snapshot, slot_tried, word_tried, success)

    def check(sa, wa, assignment):

        for (s1, p1, s2, p2) in constraints:

            if s1 == sa and s2 in assignment:

                if not is_consistent(wa, p1, assignment[s2], p2):

                    return False

            if s2 == sa and s1 in assignment:

```

```

        if not is_consistent(assignment[s1], p1, wa, p2):
            return False

    return True

def backtrack(assign, remaining):
    if not remaining:
        return True

    slot = remaining[0]

    for word in domains[slot]:
        if word in assign.values():
            continue

        if check(slot, word, assign):
            assign[slot] = word
            steps.append((dict(assign), slot, word, True))

            if backtrack(assign, remaining[1:]):
                return True

            steps.append((dict(assign), slot, word, False))

            del assign[slot]

    return False

success = backtrack(assignment, slot_names)

return assignment if success else None, steps, constraints

def fill_grid(grid, slots, assignment):
    g = copy.deepcopy(grid)

    for slot, word in assignment.items():
        for idx, (r, c) in enumerate(slots[slot]["cells"]):
            g[r][c] = word[idx]

```

```

    return g

# =====
#  TERMINAL OUTPUT
#  =====

def print_results(slots, assignment, steps, wordlist):

    total = sum(len(s["cells"]) for s in slots.values())

    print()

    print("+" + "=" * 62 + "+")

    print("|    CSP -- CROSSWORD PUZZLE SOLVER                                |")

    print("+" + "=" * 62 + "+")

    print(f"|    Grid Slots      : {len(slots):<44}|")

    print(f"|    Word List       : {len(wordlist)} words{'':<38}|")

    print(f"|    Steps taken    : {len(steps):<44}|")

    print(f"|    Status         : {'SOLVED ✓' if assignment else 'NO SOLUTION ✗':<44}|")

    print("+" + "=" * 62 + "+")

    if assignment:

        print(f"\n  {'Slot':<14} {'Word':<12} {'Direction'}")

        print("  " + "-" * 40)

        for slot, word in sorted(assignment.items()):

            direction = "ACROSS" if "ACROSS" in slot else "DOWN"

            print(f"  {slot:<14} {word:<12} {direction}")

    print()

    print("  Step-by-step Backtracking (first 15 steps):")

    print(f"  {'Step':<6} {'Slot':<14} {'Word Tried':<12} {'Result'}")

```

```

print(" " + "-" * 50)

for i, (_, slot, word, ok) in enumerate(steps[:15], 1):
    print(f" {i:<6} {slot:<14} {word:<12} {'✓ Assigned' if ok else '✗ Backtrack'}")

print()

# =====
# VISUALIZATION
# =====

def draw_crossword_grid(ax, grid, slots, assignment, title="Grid", show_numbers=True):
    rows = len(grid)
    cols = len(grid[0])
    ax.set_xlim(0, cols)
    ax.set_ylim(0, rows)
    ax.set_aspect("equal")
    ax.axis("off")
    ax.set_facecolor(CLR_PANEL)
    ax.set_title(title, color=CLR_ACCENT, fontsize=9, fontfamily="monospace", pad=4)

    # Number assignments
    slot_numbers = {}
    numbered_cells = set()
    for slot in slots:
        cell = slots[slot]["cells"][0]
        if cell not in numbered_cells:
            num = slot.split("-")[0]
            slot_numbers[cell] = num
            numbered_cells.add(cell)

```

```

for r in range(rows):
    for c in range(cols):
        x, y = c, rows - 1 - r
        val = grid[r][c]
        if val == -1:
            ax.add_patch(FancyBboxPatch(
                (x + 0.03, y + 0.03), 0.94, 0.94,
                boxstyle="square,pad=0", facecolor=CLR_CELL_BLOCK,
                edgecolor="#000000", linewidth=0.5, zorder=2
            ))
        else:
            letter = val if isinstance(val, str) else ""
            ax.add_patch(FancyBboxPatch(
                (x + 0.03, y + 0.03), 0.94, 0.94,
                boxstyle="square,pad=0",
                facecolor=CLR_CELL_EMPTY if not letter else "#1A2F1A",
                edgecolor=CLR_BORDER, linewidth=0.8, zorder=2
            ))
            if letter:
                ax.text(x + 0.5, y + 0.42, letter,
                        ha="center", va="center", fontsize=13, fontweight="bold",
                        color=CLR_GREEN, fontfamily="monospace", zorder=4)
            if show_numbers and (r, c) in slot_numbers:
                ax.text(x + 0.12, y + 0.88, slot_numbers[(r, c)],
                        ha="left", va="top", fontsize=5,
                        color=CLR_YELLOW, fontfamily="monospace", zorder=4)

```

```

def visualize(slots, wordlist, assignment, steps, constraints):

    fig = plt.figure(figsize=(18, 11), facecolor=CLR_BG)

    fig.suptitle(

        "CSP - Crossword Puzzle Solver\n"

        "Technique: Backtracking + Constraint Propagation",

        color=CLR_HEADER, fontsize=15, fontweight="bold",

        fontfamily="monospace", y=0.98

    )

    gs = gridspec.GridSpec(2, 4, figure=fig,

                           hspace=0.5, wspace=0.4,

                           left=0.03, right=0.97, top=0.90, bottom=0.06)

    # — Panel 1: Empty Grid —————

    ax1 = fig.add_subplot(gs[0, 0])

    draw_crossword_grid(ax1, DEFAULT_GRID, slots, {}, "Empty Grid")

    # — Panel 2: Solved Grid —————

    ax2 = fig.add_subplot(gs[0, 1])

    solved_grid = fill_grid(DEFAULT_GRID, slots, assignment) if assignment else DEFAULT_GRID

    draw_crossword_grid(ax2, solved_grid, slots, assignment, "Solved Grid ✓")

    # — Panel 3: Slot Info Table —————

    ax3 = fig.add_subplot(gs[0, 2])

    ax3.set_facecolor(CLR_PANEL)

    ax3.axis("off")

    ax3.set_title("Slot Assignments", color=CLR_ACCENT, fontsize=9, fontfamily="monospace",

pad=4)

    table_data = []

```

```

for slot in sorted(slots.keys()):

    word = assignment.get(slot, "---") if assignment else "---"

    direction = "→" if "ACROSS" in slot else "↓"

    table_data.append([slot, direction, str(slots[slot]["length"]), word])

tbl = ax3.table(

    cellText=table_data,

    colLabels=["Slot", "Dir", "Len", "Word"],

    cellLoc="center", loc="center",

    bbox=[0.0, 0.05, 1.0, 0.85]

)

tbl.auto_set_font_size(False)

tbl.set_fontsize(9)

for (r, c), cell in tbl.get_celld().items():

    cell.set_facecolor(CLR_BG if r == 0 else CLR_PANEL)

    cell.set_edgecolor(CLR_BORDER)

    wrd_col = CLR_GREEN if (r > 0 and table_data[r-1][3] != "---") else CLR_TEXT

    cell.set_text_props(

        color=CLR_ACCENT if r == 0 else (CLR_GREEN if c == 3 and r > 0 and
table_data[r-1][3] != "---" else CLR_TEXT),

        fontfamily="monospace",

        fontweight="bold" if r == 0 else "normal"

    )

# — Panel 4: Stats —————

ax4 = fig.add_subplot(gs[0, 3])

ax4.set_facecolor(CLR_PANEL)

ax4.axis("off")

ax4.set_title("CSP Statistics", color=CLR_ACCENT, fontsize=9, fontfamily="monospace",
pad=4)

```

```

stats = [

    ("Variables",    f"{len(slots)} slots"),

    ("Domain Size",  f"{len(wordlist)} words"),

    ("Constraints",  f"{len(constraints)} shared cells"),

    ("Steps",        str(len(steps))),

    ("Status",       "SOLVED ✓" if assignment else "FAILED ✗"),

]

for i, (k, v) in enumerate(stats):

    y = 0.88 - i * 0.16

    ax4.text(0.05, y, f"► {k}:", color=CLR_DIM, fontsize=9,

            fontfamily="monospace", transform=ax4.transAxes)

    clr = CLR_GREEN if "SOLVED" in v else (CLR_RED if "FAILED" in v else CLR_TEXT)

    ax4.text(0.58, y, v, color=clr, fontsize=9, fontweight="bold",

            fontfamily="monospace", transform=ax4.transAxes)

# — Panel 5: Backtracking Steps Bar Chart —————

ax5 = fig.add_subplot(gs[1, :2])

ax5.set_facecolor(CLR_PANEL)

ax5.set_title("Backtracking Steps per Slot", color=CLR_ACCENT,

            fontsize=9, fontfamily="monospace", pad=4)

slot_attempts = {s: 0 for s in slots}

slot_backtracks = {s: 0 for s in slots}

for _, slot, _, ok in steps:

    if ok:

        slot_attempts[slot] = slot_attempts.get(slot, 0) + 1

    else:

        slot_backtracks[slot] = slot_backtracks.get(slot, 0) + 1

```

```

snames = list(slots.keys())

x      = range(len(snames))

atts   = [slot_attempts.get(s, 0) for s in snames]

backs  = [slot_backtracks.get(s, 0) for s in snames]

ax5.bar([i - 0.2 for i in x], atts, width=0.38, color=CLR_GREEN, label="Assigned",
alpha=0.85)

ax5.bar([i + 0.2 for i in x], backs, width=0.38, color=CLR_RED, label="Backtrack",
alpha=0.85)

ax5.set_xticks(list(x))

ax5.set_xticklabels(snames, rotation=30, ha="right", fontsize=7.5,
                    color=CLR_TEXT, fontfamily="monospace")

ax5.set_ylabel("Count", color=CLR_DIM, fontsize=8, fontfamily="monospace")

ax5.tick_params(colors=CLR_DIM)

ax5.set_facecolor(CLR_PANEL)

ax5.spines[:].set_color(CLR_BORDER)

ax5.legend(facecolor=CLR_BG, edgecolor=CLR_BORDER,
           labelcolor=CLR_TEXT, fontsize=8)

for label in ax5.get_yticklabels():
    label.set_color(CLR_DIM)
    label.set_fontfamily("monospace")

# — Panel 6: Constraint Arc Graph —————
ax6 = fig.add_subplot(gs[1, 2:])

ax6.set_facecolor(CLR_PANEL)

ax6.set_title("Constraint Arc Graph (Slot Intersections)",
              color=CLR_ACCENT, fontsize=9, fontfamily="monospace", pad=4)

ax6.set_xlim(-1.5, 1.5)

ax6.set_ylim(-1.5, 1.5)

```

```

ax6.axis("off")

import math

snames = list(slots.keys())

n = len(snames)

pos = {}

for i, s in enumerate(snames):

    angle = 2 * math.pi * i / n - math.pi / 2

    pos[s] = (math.cos(angle), math.sin(angle))

drawn_edges = set()

for (sa, pa, sb, pb) in constraints:

    key = tuple(sorted([sa, sb]))

    if key not in drawn_edges:

        drawn_edges.add(key)

        x1, y1 = pos[sa]

        x2, y2 = pos[sb]

        ax6.annotate("", xy=(x2, y2), xytext=(x1, y1),

                      arrowprops=dict(

                          arrowstyle="<->",

                          color=CLR_ACCENT, lw=1.0, alpha=0.5

                      ))

for s in snames:

    x, y = pos[s]

    clr = CLR_ACROSS if "ACROSS" in s else CLR_DOWN

    ax6.add_patch(plt.Circle((x, y), 0.12, color=CLR_PANEL, ec=clr, lw=2, zorder=3))

    ax6.text(x, y, s.replace("-", "\n"), ha="center", va="center",

            fontsize=5.5, color=CLR_TEXT, fontfamily="monospace", zorder=4)

```

```

# Legend

ax6.add_patch(plt.Circle((-1.3, -1.3), 0.05, color=CLR_PANEL, ec=CLR_ACROSS, lw=2))

ax6.text(-1.15, -1.3, "ACROSS", va="center", fontsize=7, color=CLR_ACROSS,
fontfamily="monospace")

ax6.add_patch(plt.Circle((-0.3, -1.3), 0.05, color=CLR_PANEL, ec=CLR_DOWN, lw=2))

ax6.text(-0.15, -1.3, "DOWN", va="center", fontsize=7, color=CLR_DOWN,
fontfamily="monospace")


# Status banner

stat_clr = CLR_GREEN if assignment else CLR_RED

stat_txt = (f"✓ CROSSWORD SOLVED - {len(slots)} slots filled | {len(steps)}
backtracking steps"

            if assignment else f"✗ NO SOLUTION FOUND - {len(steps)} steps
attempted")

fig.text(0.5, 0.013, stat_txt, ha="center", va="bottom",

        fontsize=10, fontweight="bold", fontfamily="monospace",

        color=stat_clr,

        bbox=dict(boxstyle="round,pad=0.4",

                    facecolor="#0D2818" if assignment else "#2D1B1B",

                    edgecolor=stat_clr, lw=1.5))

plt.savefig(OUTPUT_FILE, dpi=160, bbox_inches="tight", facecolor=CLR_BG)

print(f"\n [INFO] Visualization saved -> {OUTPUT_FILE}")

plt.show()


# =====

# USER INPUT

# =====

```

```

def get_input():

    print()

    print("+" + "=" * 62 + "+")

    print("|    CSP  --  Crossword Puzzle Solver                                |")

    print("|    Slots: 1-ACROSS, 3-ACROSS, 5-ACROSS, 1-DOWN, 2-DOWN, 3-DOWN|")

    print("+" + "=" * 62 + "+")

    print()

    print("  Press Enter to use default 5x5 grid + built-in word list")

    choice = input("  or type  custom  to supply your own word list : ").strip().lower()

    if choice == "custom":

        print("\n  Enter words separated by spaces (5-letter words only for default grid):")

        raw = input("  Words : ").strip().upper().split()

        return DEFAULT_SLOTS, raw if raw else DEFAULT_WORDLIST

    return DEFAULT_SLOTS, DEFAULT_WORDLIST

# =====

#  MAIN

# =====

def main():

    print()

    print("+" + "=" * 62 + "+")

    print("|          CSP  --  Crossword Puzzle Solver                                |")

    print("|          Technique : Backtracking + Constraint Propagation                |")

    print("+" + "=" * 62 + "+")

```

```

slots, wordlist = get_input()

constraints = get_constraints(slots)


print(f"\n  Grid    : 5 × 5")

print(f"  Slots   : {len(slots)} ( {', '.join(slots.keys())} )")

print(f"  Words    : {len(wordlist)}")

print(f"  Arcs     : {len(constraints)}")

print(f"\n  Solving ... \n")


assignment, steps, constraints = solve_crossword(slots, wordlist)


print_results(slots, assignment, steps, wordlist)

visualize(slots, wordlist, assignment, steps, constraints)


if __name__ == "__main__":

    main()

```

**Output :**

```
(dl-env) sarveshmde@Sarveshs-MacBook-Air: Artificial Intelligence % python Assignment-06.py  
| Slots: 1-ACROSS, 3-ACROSS, 5-ACROSS, 1-DOWN, 2-DOWN, 3-DOWN |  
+=====+
```

Press Enter to use default 5x5 grid + built-in word list  
or type custom to supply your own word list :

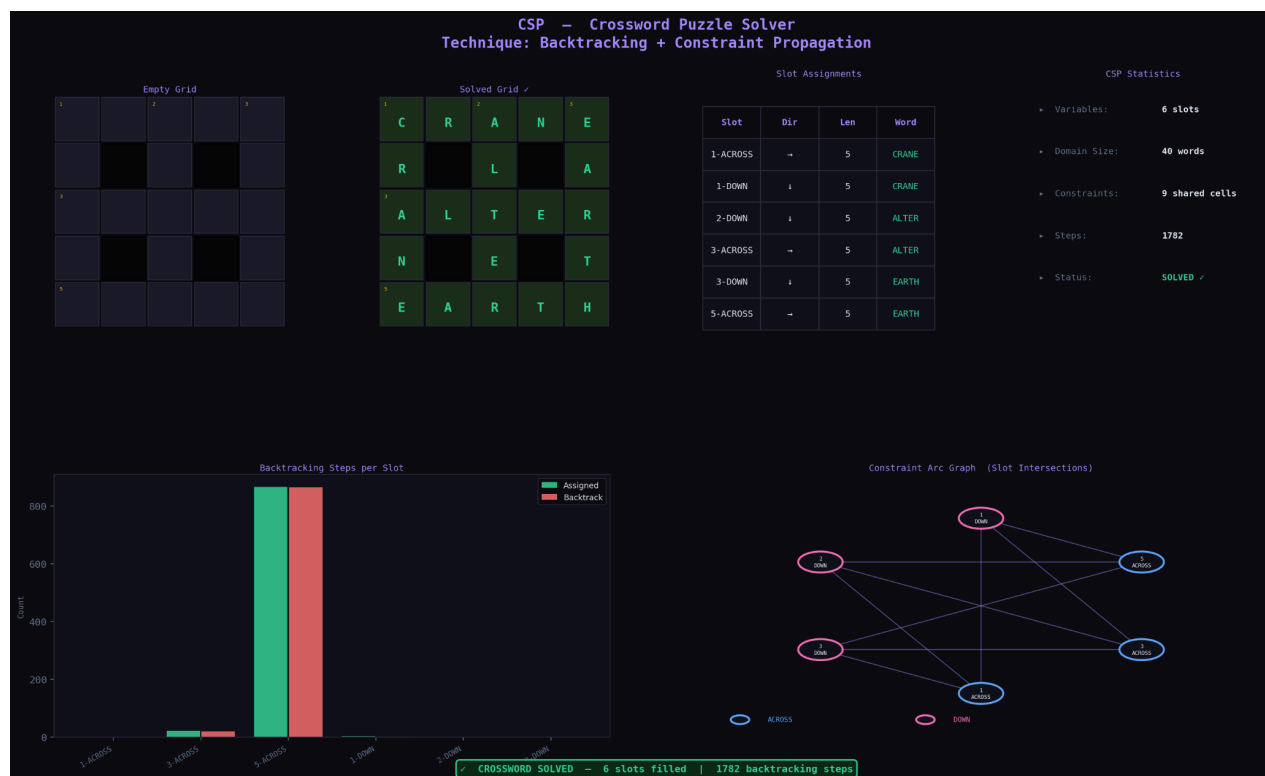
Grid : 5 x 5  
Slots : 6 ( 1-ACROSS, 3-ACROSS, 5-ACROSS, 1-DOWN, 2-DOWN, 3-DOWN )  
Words : 20  
Arcs : 9

Solving ...

```
+=====+  
| CSP -- CROSSWORD PUZZLE SOLVER |  
+=====+  
| Grid Slots : 6 |  
| Word List : 20 words |  
| Steps taken : 14948 |  
| Status : NO SOLUTION x |  
+=====+
```

Step-by-step Backtracking (first 15 steps):

| Step | Slot     | Word Tried | Result      |
|------|----------|------------|-------------|
| 1    | 1-ACROSS | CRANE      | ✓ Assigned  |
| 2    | 3-ACROSS | SHARE      | ✓ Assigned  |
| 3    | 5-ACROSS | ALTAR      | ✓ Assigned  |
| 4    | 5-ACROSS | ALTAR      | x Backtrack |
| 5    | 5-ACROSS | CASTE      | ✓ Assigned  |
| 6    | 5-ACROSS | CASTE      | x Backtrack |
| 7    | 5-ACROSS | RESTS      | ✓ Assigned  |
| 8    | 5-ACROSS | RESTS      | x Backtrack |
| 9    | 5-ACROSS | CHORE      | ✓ Assigned  |
| 10   | 5-ACROSS | CHORE      | x Backtrack |
| 11   | 5-ACROSS | ANGER      | ✓ Assigned  |
| 12   | 5-ACROSS | ANGER      | x Backtrack |
| 13   | 5-ACROSS | NASAL      | ✓ Assigned  |
| 14   | 5-ACROSS | NASAL      | x Backtrack |
| 15   | 5-ACROSS | TERSE      | ✓ Assigned  |



## c. Map coloring problem

### Theory:

Map coloring is a classic CSP where regions on a map must be colored such that no two adjacent regions have the same color. The goal is to use the minimum number of colors while satisfying constraints.

Constraints involved:

- Adjacent regions cannot have the same color.
- Only a limited set of colors is available.

This problem is commonly solved using **backtracking** or **graph coloring techniques**, where the map is represented as a graph.

### Algorithm:

Represent the map as a graph:

- Regions → Nodes
- Borders → Edges

Define a set of colors.

Assign colors to each region.

For each region:

- Try assigning a color.
- Check if adjacent regions have different colors.

If valid:

- Move to next region

If not valid:

- Try another color

If no color works, backtrack.

Repeat until all regions are colored.

Display the color assignment.

## Code:

```
"""
=====

Constraint Satisfaction Problem (CSP)

Problem : MAP COLORING

Classic: Australia / Custom Region Map

Technique: Backtracking + Forward Checking

=====
"""

import os
import copy
import math
import matplotlib.pyplot as plt
```

```

import matplotlib.gridspec as gridspec

import matplotlib.patches as mpatches

from matplotlib.patches import FancyBboxPatch, FancyArrowPatch


SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
OUTPUT_FILE = os.path.join(SCRIPT_DIR, "csp_map_coloring.png")


# — Colour palette —

CLR_BG      = "#0C0C14"

CLR_PANEL   = "#131320"

CLR_ACCENT  = "#F59E0B"

CLR_GREEN   = "#10B981"

CLR_RED     = "#EF4444"

CLR_TEXT    = "#F1F5F9"

CLR_DIM     = "#64748B"

CLR_BORDER  = "#1E1E30"

CLR_HEADER  = "#F59E0B"


# Map region fill colors (CSP domain)

DOMAIN_COLORS = {

    "Red":    "#EF4444",

    "Green":  "#10B981",

    "Blue":   "#3B82F6",

    "Yellow": "#F59E0B",

}


# Region polygon positions for plotting (centroid x, y)

AUSTRALIA_POS = {

    "WA":     (1.2, 3.5),

```

```

    "NT": (3.0, 4.2),

    "SA": (3.2, 2.8),

    "QLD": (5.0, 4.2),

    "NSW": (5.2, 2.8),

    "VIC": (4.6, 1.8),

    "TAS": (4.8, 0.7),
}

AUSTRALIA_ADJ = {

    "WA": ["NT", "SA"],

    "NT": ["WA", "SA", "QLD"],

    "SA": ["WA", "NT", "QLD", "NSW", "VIC"],

    "QLD": ["NT", "SA", "NSW"],

    "NSW": ["QLD", "SA", "VIC"],

    "VIC": ["SA", "NSW"],

    "TAS": [],

}

# =====

# CSP SOLVER

# =====

def is_valid(region, color, assignment, adjacency):

    for neighbor in adjacency.get(region, []):

        if assignment.get(neighbor) == color:

            return False

    return True

```

```

def solve_map_coloring(regions, adjacency, colors):
    """
    Backtracking CSP solver with Forward Checking.
    Returns assignment dict and search steps.
    """
    assignment = {}
    steps = [] # (region, color_tried, ok, assignment_snapshot)
    domains = {r: list(colors) for r in regions}

    def forward_check(region, color, domains):
        new_domains = copy.deepcopy(domains)
        for neighbor in adjacency.get(region, []):
            if neighbor not in assignment:
                if color in new_domains[neighbor]:
                    new_domains[neighbor].remove(color)
                if not new_domains[neighbor]:
                    return None # Domain wipeout
        return new_domains

    def backtrack(assign, unassigned, domains):
        if not unassigned:
            return True

        # MRV (Minimum Remaining Values) heuristic
        region = min(unassigned, key=lambda r: len(domains[r]))

        for color in domains[region]:
            if is_valid(region, color, assign, adjacency):

```

```

        assign[region] = color

        steps.append((region, color, True, dict(assign)))

        new_domains = forward_check(region, color, domains)

        if new_domains is not None:

            remaining = [r for r in unassigned if r != region]

            if backtrack(assign, remaining, new_domains):

                return True

        steps.append((region, color, False, dict(assign)))

        del assign[region]

    return False

success = backtrack(assignment, list(regions), domains)

return assignment if success else None, steps

# =====
#  TERMINAL OUTPUT
#  =====

def print_results(regions, adjacency, colors, assignment, steps):

    print()

    print("+" + "=" * 62 + "+")

    print("|   CSP -- MAP COLORING SOLVER                                     |")

    print("+" + "=" * 62 + "+")

    print(f"|   Regions       : {len(regions):<46}|")

    print(f"|   Colors        : {len(colors):<46}|")

```

```

print(f"| Steps      : {len(steps):<46}|")

print(f"| Status       : {'SOLVED ✓' if assignment else 'NO SOLUTION ✗':<46}|")

print("+" + "=" * 62 + "+")

if assignment:

    print(f"\n  {'Region':<10} {'Color':<10} {'Neighbors':<30} {'Conflict?'}")

    print("  " + "=" * 60)

    for region in sorted(assignment.keys()):

        color = assignment[region]

        neighbors = adjacency.get(region, [])

        neighbor_colors = [f"{n}={assignment.get(n,'?')}" for n in neighbors]

        conflict = any(assignment.get(n) == color for n in neighbors)

        nc_str = ", ".join(neighbor_colors) if neighbor_colors else "None"

        status = "✗ CONFLICT" if conflict else "✓ OK"

        print(f"  {region:<10} {color:<10} {nc_str:<30} {status}")

print(f"\n Step-by-step (first 20):")

print(f"  {'Step':<6} {'Region':<8} {'Color':<10} {'Result'}")

print("  " + "-" * 38)

for i, (reg, col, ok, _) in enumerate(steps[:20], 1):

    print(f"  {i:<6} {reg:<8} {col:<10} {'✓ Placed' if ok else '✗ Backtrack'}")

print()

# =====
#  VISUALIZATION
#  =====

def draw_graph(ax, regions, adjacency, assignment, pos, title="Map Graph", colors_map=None):

```

```

ax.set_facecolor(CLR_PANEL)

ax.axis("off")

ax.set_title(title, color=CLR_ACCENT, fontsize=9, fontfamily="monospace", pad=4)

all_x = [p[0] for p in pos.values()]
all_y = [p[1] for p in pos.values()]

margin = 1.0

ax.set_xlim(min(all_x) - margin, max(all_x) + margin)

ax.set_ylim(min(all_y) - margin, max(all_y) + margin)

# Draw edges

drawn = set()

for region, neighbors in adjacency.items():

    for nb in neighbors:

        key = tuple(sorted([region, nb]))

        if key not in drawn:

            drawn.add(key)

            x1, y1 = pos[region]

            x2, y2 = pos[nb]

            ax.plot([x1, x2], [y1, y2], color=CLR_BORDER, lw=1.5, zorder=1)

# Draw nodes

for region in regions:

    x, y = pos[region]

    color = assignment.get(region) if assignment else None

    fill = DOMAIN_COLORS.get(color, CLR_PANEL) if color else CLR_PANEL

    edge = DOMAIN_COLORS.get(color, CLR_ACCENT) if color else CLR_DIM

    circle = plt.Circle((x, y), 0.35, color=fill, ec=edge, lw=2.5, zorder=3)

```

```

    ax.add_patch(circle)

    ax.text(x, y, region, ha="center", va="center",
            fontsize=9, fontweight="bold", color=CLR_TEXT,
            fontfamily="monospace", zorder=4)

def visualize(regions, adjacency, colors, assignment, steps):
    fig = plt.figure(figsize=(18, 12), facecolor=CLR_BG)

    fig.suptitle(
        "CSP - Map Coloring Problem (Australia)\n"
        "Technique: Backtracking + Forward Checking + MRV Heuristic",
        color=CLR_HEADER, fontsize=15, fontweight="bold",
        fontfamily="monospace", y=0.98
    )

    gs = gridspec.GridSpec(2, 4, figure=fig,
                           hspace=0.5, wspace=0.4,
                           left=0.03, right=0.97, top=0.90, bottom=0.06)

    # --- Panel 1: Uncolored Graph ---
    ax1 = fig.add_subplot(gs[0, 0:2])

    draw_graph(ax1, regions, adjacency, {}, AUSTRALIA_POS, "Constraint Graph (Uncolored)")

    # --- Panel 2: Colored Graph ---
    ax2 = fig.add_subplot(gs[0, 2:4])

    draw_graph(ax2, regions, adjacency, assignment, AUSTRALIA_POS, "Solved Map Coloring ✓")

    # Add color legend to solved graph
    legend_handles = [

```

```

    mpatches.Patch(facecolor=v, edgecolor=CLR_BG, label=k)

    for k, v in DOMAIN_COLORS.items()

    if k in (assignment or {}).values()

]

ax2.legend(

    handles=legend_handles,

    loc="lower right", facecolor=CLR_BG, edgecolor=CLR_BORDER,

    labelcolor=CLR_TEXT, fontsize=8

)

# — Panel 3: Assignment Table —————

ax3 =fig.add_subplot(gs[1, 0])

ax3.set_facecolor(CLR_PANEL)

ax3.axis("off")

ax3.set_title("Final Assignment", color=CLR_ACCENT, fontsize=9,

              fontfamily="monospace", pad=4)

if assignment:

    table_data = [[r, assignment[r]] for r in sorted(assignment.keys())]

    tbl = ax3.table(

        cellText=table_data,

        colLabels=["Region", "Color"],

        cellLoc="center", loc="center",

        bbox=[0.05, 0.05, 0.9, 0.85]

    )

    tbl.auto_set_font_size(False)

    tbl.set_fontsize(10)

    for (r, c), cell in tbl.get_celld().items():

        cell.set_facecolor(CLR_BG if r == 0 else CLR_PANEL)

```

```

        cell.set_edgecolor(CLR_BORDER)

        if r > 0 and c == 1:

            color_name = table_data[r - 1][1]

            cell.set_facecolor(DOMAIN_COLORS.get(color_name, CLR_PANEL) + "44")

        cell.set_text_props(

            color=CLR_ACCENT if r == 0 else CLR_TEXT,

            fontfamily="monospace",

            fontweight="bold" if r == 0 else "normal"

        )

# — Panel 4: Adjacency / Constraint Table —————

ax4 = fig.add_subplot(gs[1, 1])

ax4.set_facecolor(CLR_PANEL)

ax4.axis("off")

ax4.set_title("Adjacency Constraints", color=CLR_ACCENT, fontsize=9,

            fontfamily="monospace", pad=4)

table_data2 = []

for region in sorted(regions):

    nbrs = ", ".join(adjacency.get(region, [])) or "None"

    table_data2.append([region, nbrs])

tbl2 = ax4.table(

    cellText=table_data2,

    colLabels=["Region", "Neighbors"],

    cellLoc="left", loc="center",

    bbox=[0.0, 0.05, 1.0, 0.85]

)

tbl2.auto_set_font_size(False)

```

```

tbl2.set_fontsize(8.5)

for (r, c), cell in tbl2.get_celld().items():

    cell.set_facecolor(CLR_BG if r == 0 else CLR_PANEL)

    cell.set_edgecolor(CLR_BORDER)

    cell.set_text_props(

        color=CLR_ACCENT if r == 0 else CLR_TEXT,

        fontfamily="monospace"

    )


# — Panel 5: Steps Timeline —————

ax5 = fig.add_subplot(gs[1, 2:4])

ax5.set_facecolor(CLR_PANEL)

ax5.set_title("Backtracking Search Steps (Assign vs Backtrack per Region)",

              color=CLR_ACCENT, fontsize=9, fontfamily="monospace", pad=4)


assigns_per_region = {r: 0 for r in regions}

backtracks_per_region = {r: 0 for r in regions}

for reg, col, ok, _ in steps:

    if ok:

        assigns_per_region[reg] += 1

    else:

        backtracks_per_region[reg] += 1


x_vals = list(range(len(regions)))

region_list = sorted(regions)

a_vals = [assigns_per_region[r] for r in region_list]

b_vals = [backtracks_per_region[r] for r in region_list]


ax5.bar([i - 0.2 for i in x_vals], a_vals, width=0.38,

```

```

        color=CLR_GREEN, label="Assigned", alpha=0.9)

ax5.bar([i + 0.2 for i in x_vals], b_vals, width=0.38,

        color=CLR_RED, label="Backtrack", alpha=0.9)

ax5.set_xticks(x_vals)

ax5.set_xticklabels(region_list, fontsize=9, color=CLR_TEXT, fontfamily="monospace")

ax5.set_ylabel("Steps", color=CLR_DIM, fontsize=8, fontfamily="monospace")

ax5.set_facecolor(CLR_PANEL)

ax5.spines[:].set_color(CLR_BORDER)

ax5.tick_params(colors=CLR_DIM)

for label in ax5.get_yticklabels():

    label.set_color(CLR_DIM)

    label.set_fontfamily("monospace")

ax5.legend(facecolor=CLR_BG, edgecolor=CLR_BORDER, labelcolor=CLR_TEXT, fontsize=8)

# Color domain swatches

swatch_x = 0.01

for color, hex_c in DOMAIN_COLORS.items():

    ax5.add_patch(mpatches.FancyBboxPatch(

        (swatch_x, -0.5), 0.6, 0.4,

        boxstyle="round,pad=0.05", transform=ax5.transData,

        facecolor=hex_c, edgecolor=CLR_BG, linewidth=1, zorder=5

    ))

    ax5.text(swatch_x + 0.3, -0.28, color[0], ha="center", va="center",

            fontsize=7, fontweight="bold", color="white",

            fontfamily="monospace", zorder=6)

    swatch_x += 0.8

# Status banner

```

```

stat_clr = CLR_GREEN if assignment else CLR_RED

stat_txt = (f"✓ MAP COLORED - {len(assignment)} regions | "
            f"{len(colors)} colors used | {len(steps)} backtracking steps"
            if assignment else f"✗ NO VALID COLORING FOUND - {len(steps)} steps")

fig.text(0.5, 0.013, stat_txt, ha="center", va="bottom",
        fontsize=10, fontweight="bold", fontfamily="monospace",
        color=stat_clr,
        bbox=dict(boxstyle="round,pad=0.4",
                  facecolor="#0D2818" if assignment else "#2D1B1B",
                  edgecolor=stat_clr, lw=1.5))

plt.savefig(OUTPUT_FILE, dpi=160, bbox_inches="tight", facecolor=CLR_BG)

print(f"\n [INFO] Visualization saved -> {OUTPUT_FILE}")

plt.show()

# =====
# USER INPUT
# =====

def get_input():
    print()

    print("+" + "=" * 62 + "+")

    print("| CSP -- Map Coloring Solver |")

    print("| Default: Australia Map (7 regions, 3 colors) |")

    print("+" + "=" * 62 + "+")

    print()

    print(" Press Enter to use Australia map")

    choice = input(" or type custom to add extra colors/regions : ").strip().lower()

```

```

if choice == "custom":

    n_colors = input("\n  Number of colors to use [2-4] (default 3): ").strip()

    try:

        n_colors = max(2, min(4, int(n_colors)))

    except ValueError:

        n_colors = 3

    colors = list(DOMAIN_COLORS.keys())[:n_colors]

    return list(AUSTRALIA_ADJ.keys()), AUSTRALIA_ADJ, colors

return list(AUSTRALIA_ADJ.keys()), AUSTRALIA_ADJ, ["Red", "Green", "Blue"]

# =====

#  MAIN

# =====

def main():

    print()

    print("+" + "=" * 62 + "+")

    print("|      CSP -- Map Coloring Problem Solver      |")

    print("|      Technique : Backtracking + Forward Checking + MRV      |")

    print("+" + "=" * 62 + "+")

    regions, adjacency, colors = get_input()

    print(f"\n  Regions : {' , '.join(regions)}")

    print(f"  Colors  : {' , '.join(colors)}")

    print(f"\n  Solving ... \n")

```

```

assignment, steps = solve_map_coloring(regions, adjacency, colors)

print_results(regions, adjacency, colors, assignment, steps)

visualize(regions, adjacency, colors, assignment, steps)

if __name__ == "__main__":

    main()

```

## Output :

```

(ai_env) sarveshmac@Sarveshs-MacBook-Air Artificial Intelligence % python Assignment-06.py

Press Enter to use Australia map
or type custom to add extra colors/regions :

Regions : WA, NT, SA, QLD, NSW, VIC, TAS
Colors : Red, Green, Blue

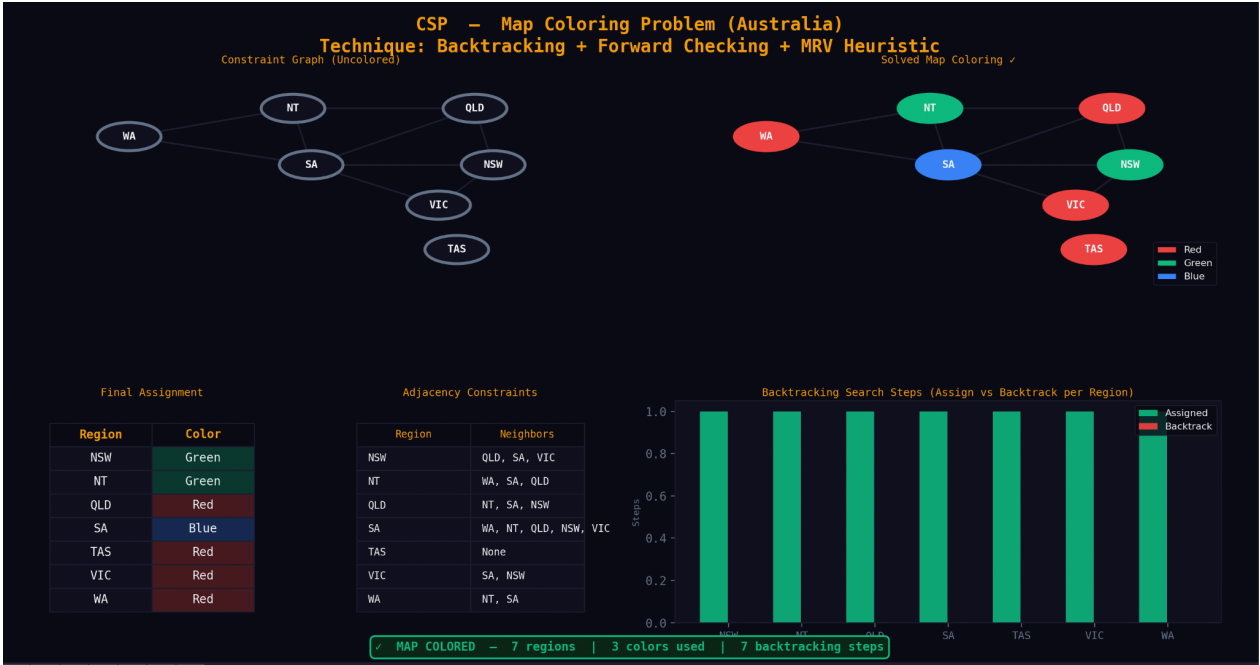
Solving ...

=====
| CSP -- MAP COLORING SOLVER |
=====
| Regions : 7 |
| Colors : 3 |
| Steps : 7 |
| Status : SOLVED ✓ |
=====

Region  Color  Neighbors  Conflict?
=====
NSW     Green   QLD=Red, SA=Blue, VIC=Red  ✓ OK
NT      Green   WA=Red, SA=Blue, QLD=Red  ✓ OK
QLD     Red     NT=Green, SA=Blue, NSW=Green ✓ OK
SA      Blue    WA=Red, NT=Green, QLD=Red, NSW=Green, VIC=Red ✓ OK
TAS     Red     None                ✓ OK
VIC     Red     SA=Blue, NSW=Green      ✓ OK
WA      Red     NT=Green, SA=Blue       ✓ OK

Step-by-step (first 20):
Step  Region  Color  Result
=====
1      WA      Red    ✓ Placed
2      NT      Green  ✓ Placed
3      SA      Blue   ✓ Placed
4      QLD     Red    ✓ Placed
5      NSW     Green  ✓ Placed
6      VIC     Red    ✓ Placed
7      TAS     Red    ✓ Placed

```



## Conclusion

In this assignment, various **Constraint Satisfaction Problems (CSPs)** such as Cryptarithmic, Crossword Puzzle, and Map Coloring were successfully studied and implemented. These problems demonstrated how complex real-world scenarios can be modeled using variables, domains, and constraints.

The use of **backtracking algorithms** proved to be an effective approach for solving CSPs by systematically exploring possible solutions and eliminating invalid ones through constraint checking. Each problem highlighted different aspects of constraint handling, such as uniqueness constraints in cryptarithmic, positional constraints in crossword puzzles, and adjacency constraints in map coloring.

Through this implementation, a deeper understanding of **problem modeling, search techniques, and optimization** was achieved. It also showed how constraints significantly reduce the search space, making problem-solving more efficient.

Overall, this assignment provided valuable practical insight into **Artificial Intelligence techniques**, particularly in solving CSPs, and strengthened logical thinking and algorithm design skills.